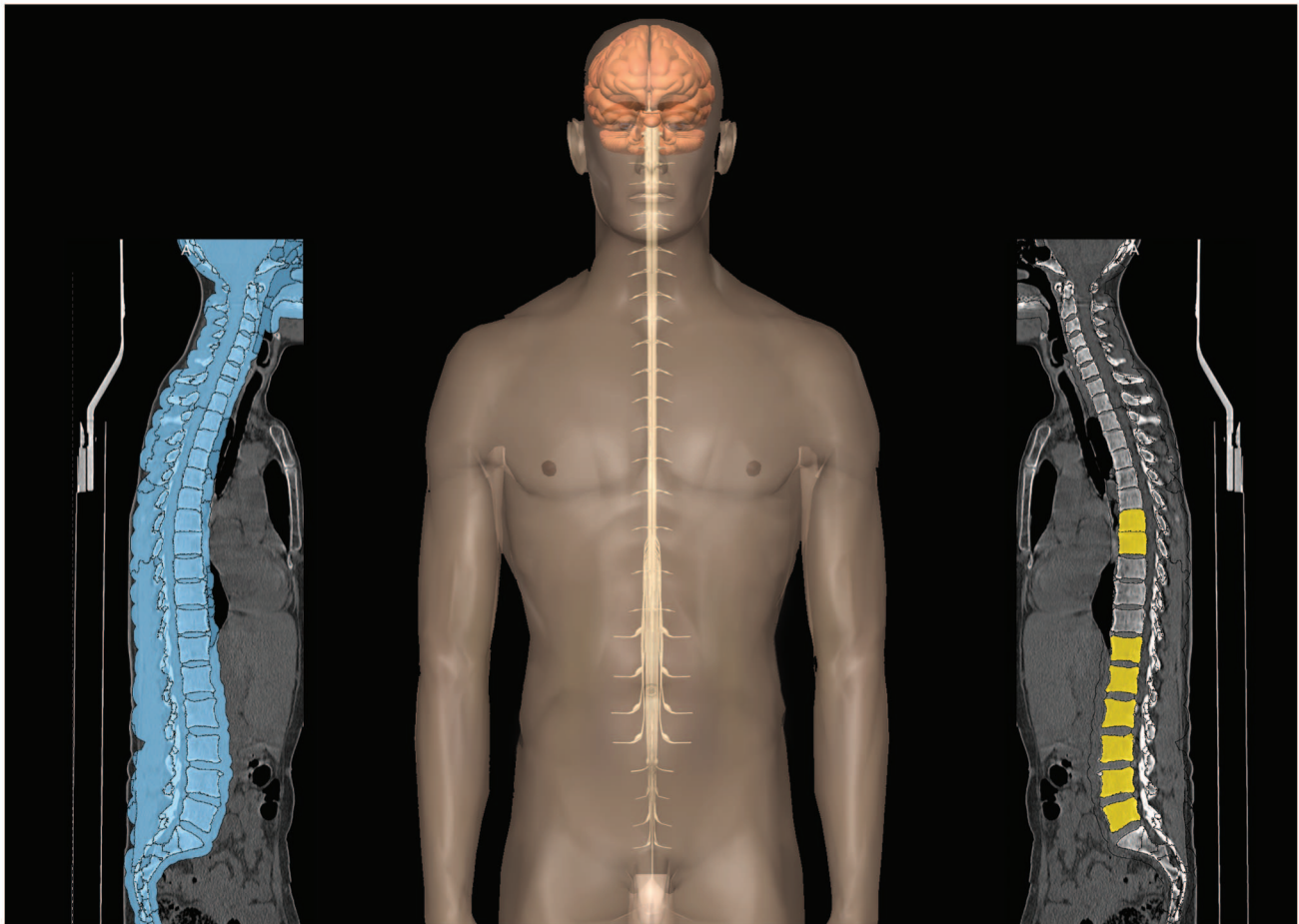


Medical Image Analysis



©TECHPOOL STUDIOS, INC.

By Felix Ritter, Tobias Boskamp, André Homeyer, Hendrik Laue,
Michael Schwier, Florian Link, and Heinz-Otto Peitgen

Since the discovery of the X-ray radiation by Wilhelm Conrad Roentgen in 1895, the field of medical imaging has developed into a huge scientific discipline. The analysis of patient data acquired by current image modalities, such as computerized tomography (CT), magnetic resonance tomography (MRT), positron emission tomography (PET), or ultrasound

(US), offers previously unattained opportunities for diagnosis, therapy planning, and therapy assessment. Medical image processing is essential to leverage this increasing amount of data and to explore and present the contained information in a way suitable for the specific medical task.

A Visual Approach

In this tutorial, we will approach the analysis and visualization of medical image data in an explorative manner. In particular, we will visually construct the image-processing algorithms using the popular graphical data-flow builder MeVisLab, which is available as a free download for

Digital Object Identifier 10.1109/MPUL.2011.942929
Date of publication: 30 November 2011

noncommercial research [1]. We felt that it could be more interesting for the reader to see and explore examples of medical image processing that go beyond simple image enhancements. The part of exploration, to inspect medical image data and experiment with image-processing pipelines, requires software that encourages this kind of visual exploration.

Image Processing

In contrast to general image processing that has primary goals, such as enhancing the esthetics of an image or creating art, the sole purpose of medical image processing is to improve the interpretability of the depicted contents. This may involve an enhancement of the image itself to increase the perception of certain features as well as the automated or manual extraction of information.

Since we can only discuss a few image analysis and visualization methods here, the following classification of the most important categories will provide a short reference for further investigation:

- ▼ *Image enhancement*: The removal of image distortions, such as noise and background inhomogeneities, as well as the enhancement of image contours and other relevant properties.
- ▼ *Image segmentation*: The identification of the contours of an anatomical structure, such as an organ, a vessel, or a tumor lesion.
- ▼ *Image registration*: The spatial transformation of one image such that it directly matches a given reference image. This is necessary, for example, in the combined visualization of images from different modalities (e.g., PET/CT).
- ▼ *Quantification*: The determination of geometrical properties of an anatomical structure (e.g., volume, diameter, and curvature) or physiological properties such as perfusion characteristics or tissue composition.
- ▼ *Visualization*: Two-dimensional (2-D) and three-dimensional (3-D) rendering of image data (e.g., volume rendering) and virtual models (e.g., surface models) of organs and other anatomical structures.
- ▼ *Computer-aided detection*: The detection and characterization of pathological structures and lesions, such as tumor lesions or vessel obstructions.

The term image is used here not only to refer to a conventional, 2-D image, for example, a radiograph. An image may also be comprised of a 3-D volume of parallel slices (e.g., a series of CT or MRT tomograms) and even a sequence of image volumes acquired over time, such as a dynamic MRT study, may be considered a four-dimensional image.

Images of volumetric imaging modalities, such as CT and MRT, are the most common source for medical image processing. Each volume is composed of volumetric image elements called *voxels* that represent a value on a regular, 3-D grid. The image values of CT images are of absolute value measured on the Hounsfield scale, a scale describing radio density. In our second example, we use knowledge about the radiation absorption of different materials to separate tissue, which has a value of around +30 Hounsfield units (HU) from air, approximately -1,000 HU.

Such a separation is the most basic form of segmentation called *thresholding*. Often, however, materials do not differ enough in radio density to be distinguishable by using thresholding alone. For instance, the HU value of blood is around +40 HU and the value of liver tissue is between +40 and +60 HU. The perception of blood-filled vascular structures within the liver would be difficult, if not impossible, given such a small difference. This is why contrast agents are important. When applied to a patient, they will change the imaging characteristics of the aimed structures. Using a contrast agent for vascular structures will increase the HU value of blood significantly above the value of the surrounding tissue (e.g., +120 HU). The contrast-enhanced blood will not only flow through the arteries and eventually arrive in the liver but also accumulate in the liver tissue itself. Hence, timing becomes very crucial for imaging certain structures.

In contrast to the Hounsfield scale of CT imaging, the MRT imaging scale has no absolute physical meaning. But like a black and a white photograph in which you can easily distinguish a darker lawn in front of a house painted in a brighter color without knowing the exact brightness of each color, MRT images will depict different tissue characteristics in a meaningful way. In most cases, relative differences in image values are sufficient for medical diagnosis and absolute values are not required. Protocols for acquiring absolute values often need multiple scans and are therefore also more time consuming.

Generally, medical images contain noise. Noise may impede the analysis of regions based on voxel values. Smoothing the image may help in this case but will also reduce the perception and detection of contours. Like with other basic image enhancement methods such as sharpening, smoothing can be based on a fundamental image operation called *convolution*. Convolution replaces each voxel value by a weighted sum of its neighbors. The weights are specified by the convolution kernel, which can be represented by a matrix. The most basic smoothing kernel replaces each voxel with the weighted average of the direct neighbors, hence will contain a value of 1 in all elements of a 3×3 matrix. More common, however, is a binomial (Gaussian-like) convolution kernel that applies more weight to less-distant neighbors.

A very common segmentation method for coherent regions of an image is region growing. Region growing combines homogeneous, neighboring voxels to form a region starting with a given, initial voxel. Accepted values of adjacent voxels may be fixed and set in the beginning or are being adapted automatically while the algorithm spreads in the vicinity. The initial set or range of accepted voxels is often derived from the value of the initial voxel.

While looking at individual slices of a volumetric image is the most common form of diagnostic reading or inspection in the preparation of a surgical intervention, 3-D visualizations may assist the analysis of spatial relations of different structures (Figure 1). Most 3-D visualizations are 2-D projections of a 3-D image displayed on a conventional screen. By adding the ability to interactively change the point of view and reveal certain structures while hiding others, 3-D visualizations become an

The sole purpose of medical image processing is to improve the interpretability of the depicted contents.

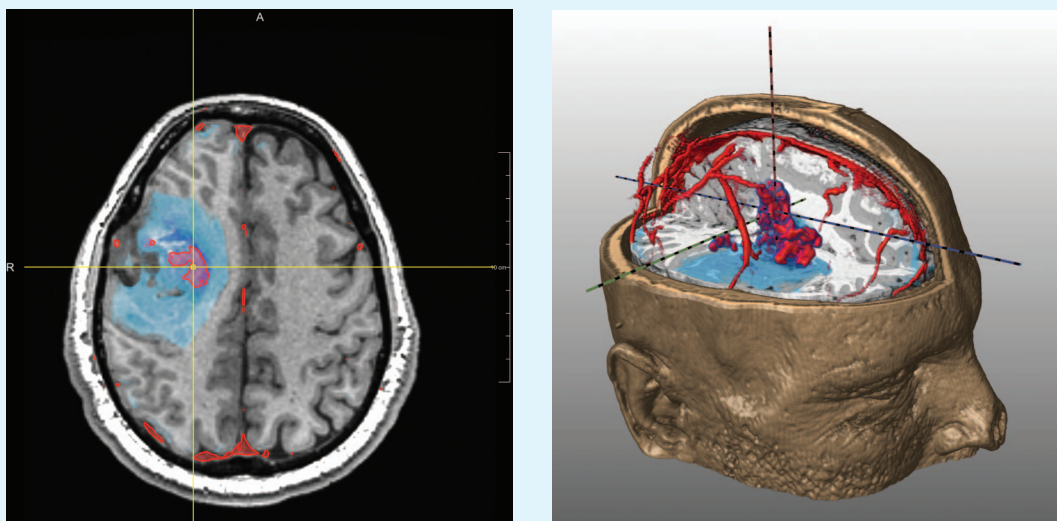


FIGURE 1 A coherent 2-D and 3-D visualizations of multimodal brain data using MeVisLab. A lesion (blue) as well as an arteriovenous malformation (red) are highlighted. The crosshair indicates the same location in both visualizations.

extremely useful tool to build up spatial understanding. There are two main 3-D visualization methods for medical image data: direct volume rendering and polygonal isosurface rendering. Direct volume rendering, as the name implies, is able to directly visualize image data. We are not required to segment structures beforehand since the whole image data are evaluated for display. Voxel values are mapped to colors and opacity values according to a transfer function. In contrast, polygonal isosurface rendering relies on the ability to extract regions from the image, which are then rendered as polygonal meshes. An isosurface represents regions of voxels of a certain value. While isosurfaces may be extracted directly from the image, this method is commonly applied to mask images of already segmented structures.

The examples discussed hereinafter will combine a number of image analysis and visualization methods to provide solutions to common medical tasks.

MeVisLab

The design and exploration of new methods in medical image processing and visualization are creative processes starting with an initial idea in need of realization and validation. In the same manner as architects sketch their ideas of a new building on a sheet of paper, making refinements on their way to the final drawing, the image-processing researcher could benefit from sketching algorithmic ideas in a visual way if this sketch can be brought to life with real medical image data.

MeVisLab provides such an environment of advanced medical image processing and visualization building blocks that can be combined visually to create new algorithms and prototypical applications, enabling the researcher or clinician to evaluate these algorithms in clinical settings. Originally designed as

an internal development tool of the German Fraunhofer MEVIS research institute and its commercial spin-off MeVis Medical Solutions, MeVisLab was made publicly available in 2004 and is currently used in numerous research and development sites worldwide.

MeVisLab includes a comprehensive, extensible library of functional components, hereinafter referred to as *modules*, for segmentation, registration, and quantitative morphological as well as functional analysis. For the visual presentation of medical data, interactive 2-D and 3-D viewing, contour, surface model, and volume representations are available (Figure 1). These modules can be combined to form module networks that implement complex image processing and visualization tasks. MeVisLab uses a graphical programming approach for the development of such

networks, allowing the developer to instantiate modules in a network and draw connections in-between to define the flow of data and control information (Figure 2). Besides the MeVisLab-native MeVis image-processing library (ML), the well-known National Library of Medicine Insight Segmentation and Registration Toolkit (ITK) as well as the Visualization ToolKit (VTK) have been integrated in a manner that encourages and allows combination of their modules.

In analogy to the concept of classes in object-oriented programming, MeVisLab allows the user to create macro modules that encapsulate a network with all its modules, connections, and parameterizations. Such macro modules may be instantiated and combined with other modules in a network, allowing one to implement even more complex networks in a structured, hierarchical, and modular way.

The parameters offered by many modules to control their internal operations can be manipulated in each module's graphical user-interface (GUI) panel (Figure 2). Connections

MeVisLab allows the user to create macro modules that encapsulate a network with all its modules, connections, and parameterizations.

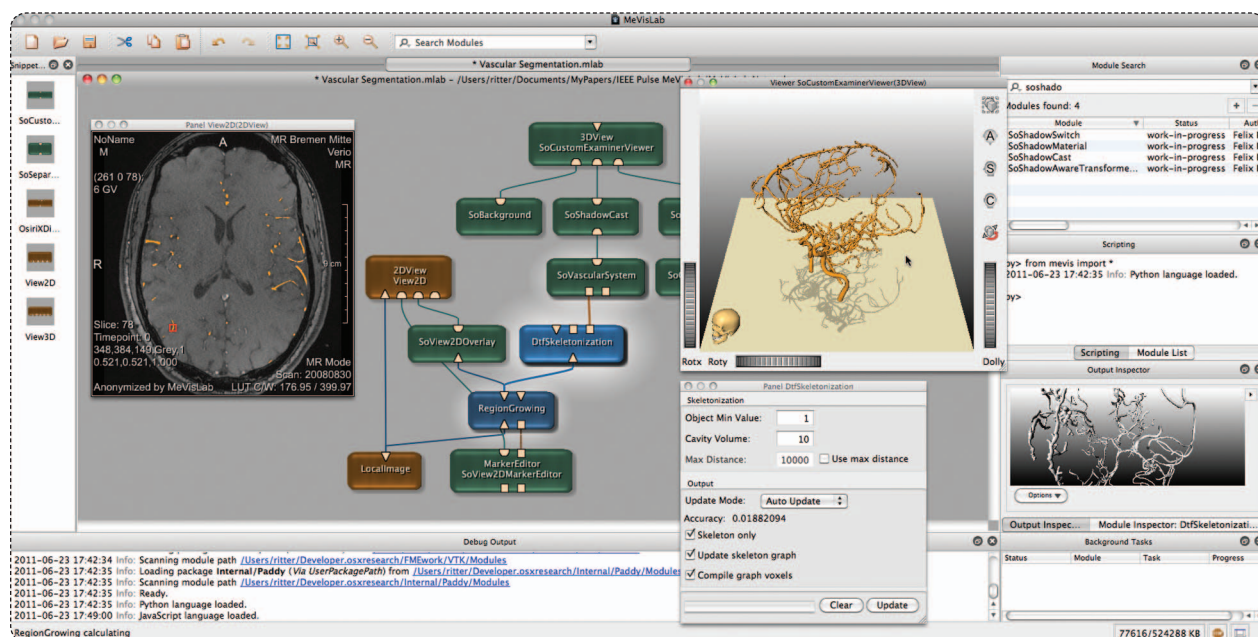


FIGURE 2 The MeVisLab visual development environment. Algorithmic building blocks displayed as boxes can be combined to form a more complex algorithm via an intuitive GUI. The parameter panel of the DtfSkeletonization algorithm is displayed to enable its configuration.

between parameters can be established to automatically transfer information between algorithms. This panel may also display result values that are generated by the module. The layout and appearance of the module panels is defined using an abstract, hierarchical scripting language. Such user-interface panels may also be defined for a complete network, giving access to only those parameters and results that are necessary for using the functionality implemented in the network.

MeVisLab offers specific modules to integrate a module network into the clinical data infrastructure. The well-accepted Digital Imaging and Communications in Medicine (DICOM) standard is used to receive image data from the imaging modalities or to transfer images from or to a clinical image data archive [picture archiving and communication system (PACS)]. DICOM tools, such as the popular OsiriX DICOM viewer, may serve as an interface for MeVisLab-based image analysis. As a result, complete software application prototypes can be created to facilitate the evaluation of image analysis or visualization method as well as dedicated workflows in a clinical setting providing valuable input for the further development of a method.

Image Processing Using MeVisLab

Two representative examples will guide our discussion on the processing of medical image data using MeVisLab. Both example networks and image data are available for download [6] and can be explored with an installation of MeVisLab [1]. The first example illustrates the segmentation of elongated structures, such as vascular systems, and their visualization in 3-D. The reconstruction of vascular structures from image data is highly useful for the diagnosis of cardiovascular dis-

MRT images will depict different tissue characteristics in a meaningful way.

eases, in particular, the detection of abnormal variations such as aneurysms. In surgery, estimating the risk of injuring the blood supply and drainage of organs is of great importance. The second example segments, quantifies, and visualizes a round lesion that is attached to a healthy tissue. Here, we would like to calculate the maximum diameter of a metastasis in the human lung that connects to the chest wall but does not still infiltrate it. The automatic calculation of the maximum diameter in 3-D provides a much more reliable indication of shrinkage or growth in follow-up examinations than the manual measurement of a diameter in just one cross-sectional image.

Segmentation and Visualization of Vascular Structures

If you like to follow the discussion of the reconstruction of vascular structures in the MeVisLab application, please open the network file `Vascular Segmentation.mlab` in [6]. In the left viewing panel of the example network, click with your mouse in one of the bright white circular spots that represent a vascular structure. Click again in another white spot to add other vascular structures (Figure 2). To remove the markers you just placed, double-click on the module `MarkerEditor` in the network window and activate the button `Delete all`. The segmented structures are indicated in an orange color in the 2-D and 3-D views.

The reconstruction of vascular structures from a volumetric data set (stack of images) involves several steps (see Figures 3 and 4):

- 1) *Preprocessing*: Gaussian smoothing of noisy image data.
- 2) *Segmentation*: Region growing using an initial seed point.

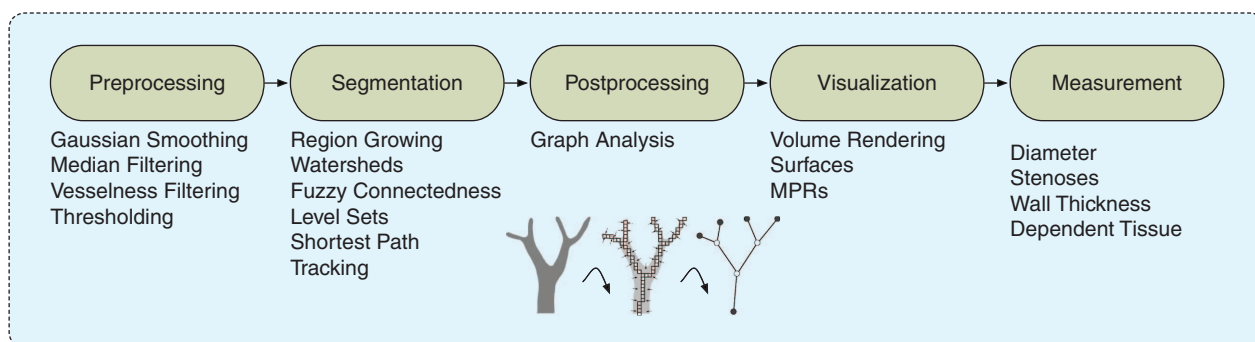


FIGURE 3 An image-processing pipeline for the visualization and quantification of vascular structures from image data.

- 3) *Postprocessing*: Analysis of the segmentation object and construction of a model-based representation.
- 4) *Visualization*: Surface rendering of the model-based representation.
- 5) *Measurement*: Quantification of dependent tissue for risk estimation in surgery.

For our first example, we omit the preprocessing step and start with the segmentation of the loaded image data. Loading image data can be done in several ways, depending on the nature of the data, but like the rest of the image-processing pipeline always involves a module. A series of DICOM images, which is the most basic form for medical images, can be retrieved from a disk using the `DirectDicomImport` module. The provided example

data have been compacted and stored as a single file, compressed 3-D data set in the MeVisLab native format. These data are accessible via the `ImageLoad` module. We use a variant of this module called `LocalImage` to automatically locate the data in the directory of our example network. This module is a good choice for demonstration networks.

In MeVisLab, we distinguish between image-processing modules and modules that perform visualization tasks because of the different underlying frameworks. Whereas image-processing modules (colored in blue) form a pipeline, for which dedicated input and output connectors are provided, visualization modules (colored in green) are connected in a hierarchical graph called a *scene graph*. This scene graph is traversed depth

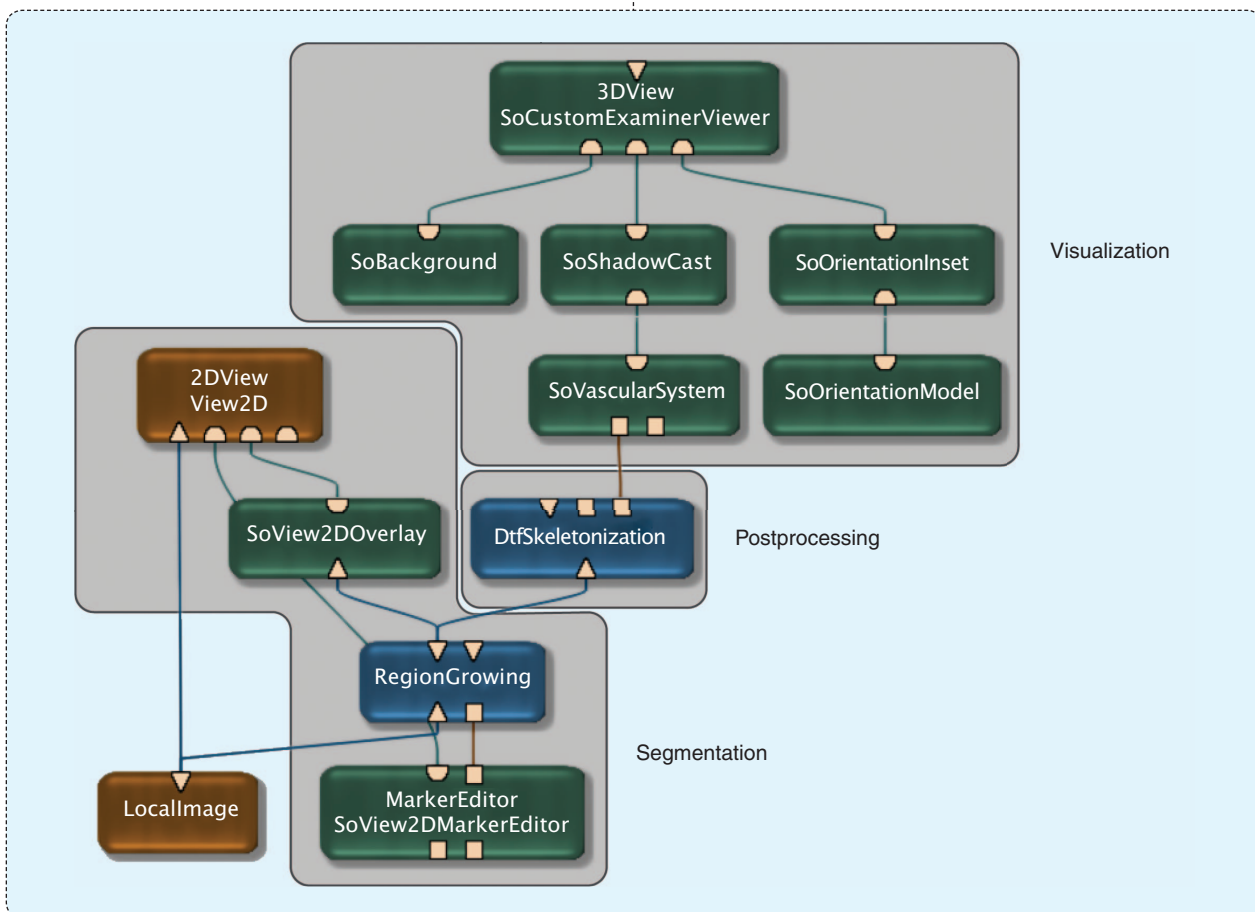


FIGURE 4 Segmentation, postprocessing, and visualization parts of the vascular reconstruction network.

first from left to right. At the root of this scene graph, we often find a viewing component displaying the visual result of the image-processing network.

Visualization and analysis often involve a separation of structures. We apply a 3-D region growing approach to the original image data to segment the vascular system. Starting with an initial voxel from the contrast-enhanced image, neighboring voxels of similar value are combined. A 2-D slice-based visualization, commonly found in medical workstations, enables the diagnostic exploration of the image set and allows the user to place a marker on an initial voxel using the mouse or trackpad. The segmented structures are highlighted on each slice to indicate the result of the region growing. Two main parameters influence the result, the lower and upper thresholds of voxel values and the initial voxel locations. Multiple markers may be placed to segment disconnected regions. The implementation tries to minimize computation by adding to or removing from already segmented regions if parameters fit.

The 3-D visualization of the segmented vascular structures supports the recognition of branching patterns as well as the inspection and analysis of their path. As outlined before, we may apply direct volume rendering and polygonal isosurface rendering. With both methods, however, small vessels tend to disappear completely. Owing to the discrete nature of the images and the segmentation mask, a model-based reconstruction algorithm that respects the thin structures of vessel trees whose diameters often grow and shrink from slice to slice will result in a much smoother visualization.

A simple but adequate model assumes that the cross section of nonpathologic vessels has a circular shape. Based on the Skeletonization algorithm with topology-preserving thinning,

an exact centerline and the radius at each voxel of the skeleton are computed. The skeletonization uses the binary segmentation image and successively erodes border voxels until a center voxel is found. A subsequent graph analysis transforms the

skeleton in a directed, acyclic graph where nodes represent branches (Figure 3). Further enhancements may involve pruning and smoothing of the graph. This model has been implemented in a MeVisLab module called *DtfSkeletonization*. It is directly connected to the *SoVascularSystem* module for the final graph-based visualization. The module, among other things, allows the user to limit the rendering of the vascular structures to a certain branching level. For

instance, this is helpful when supporting planning of surgical interventions where branching levels indicate the volume of the dependent tissue.

The example previously shown in Figure 2 adds a shadow and a 3-D depiction of the skull to the 3-D view to facilitate orientation for nonexperts. Buttons for the three main anatomical planes, axial (aka transverse), sagittal, and coronal, are placed at the right side of the 3-D view. For an occasional analysis of image data, working directly with the MeVisLab network and individual module panels might be acceptable. However, for frequent use or studies run by a different person, a more usable and comprehensible user interface (UI) will be less error prone and also more productive. MeVisLab provides UI building capabilities to hide the network complexity from the user. Whole module panels or parts thereof can be combined with viewing panels and other supporting UI components to form a custom-tailored image-processing application. Figure 5 shows the final UI for this example network.

The UI script is written in a simple UI language (see Figure 6) that references module panels or parameter fields. For instance,

MeVisLab provides UI building capabilities to hide the network complexity from the user.

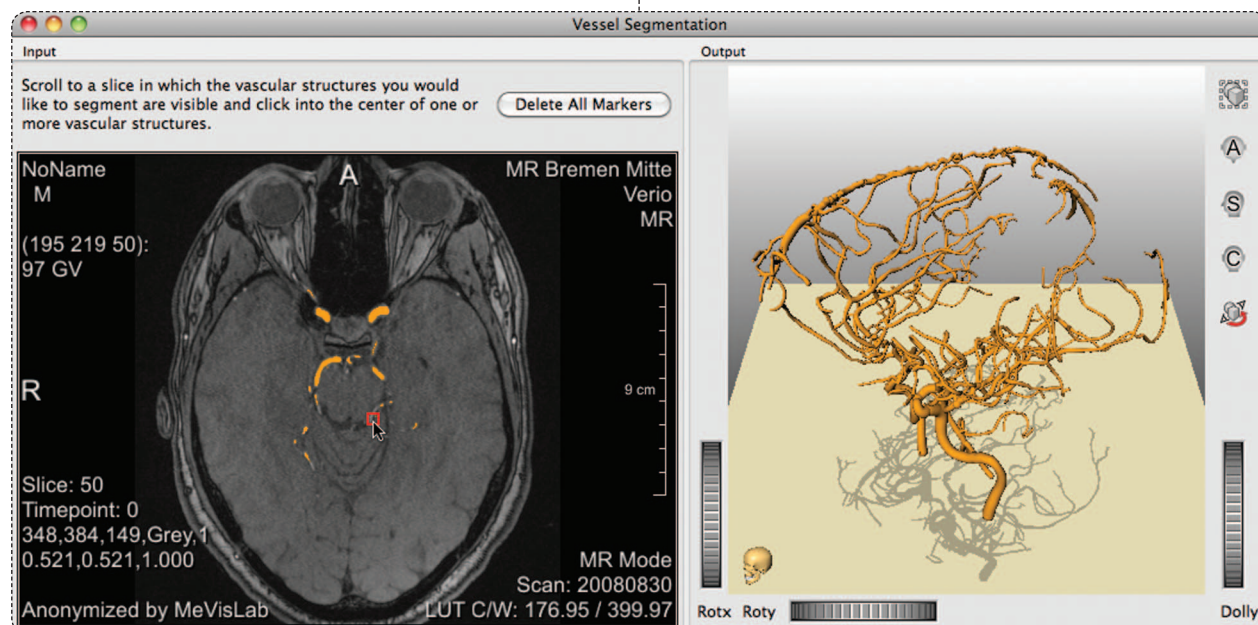


FIGURE 5 The final UI of the vascular segmentation example. The complexity of the underlying module network is hidden from the user. Choose Scripting → Start Network Script in MeVisLab to display this UI.

the Delete All Markers button in our final UI is created by referring to the parameter field `deleteAll` of the module `MarkerEditor`. `MarkerEditor` is the instance name of the `SoView2DMarkerEditor` module. The viewing panels are also referred to by the instance name but are suffixed by the special identifier `.self`. Layout components, such as boxes as well as horizontal or vertical alignment groups, help to arrange and semantically combine interface components. To create or open the UI script of a MeVisLab network, select Scripting → Edit Network Script from the menu bar. The script will open in the MeVisLab text editor, which provides syntax highlighting, context-sensitive help, and automatic completion for the modules and fields of the image-processing network. The result of the UI script can be tested at any time within MeVisLab by selecting Scripting → Start Network Script from the menu bar. If all module panels are closed and the MeVisLab visual development environment (IDE) has been minimized, the custom-designed UI of our image-processing application remains the only visible part.

Segmentation, Visualization, and Quantification of Lesions

The second example segments and computes the maximum diameter of a lung metastasis. Automatic segmentation methods are often restricted to a specific type of object and data on which they operate because of need for heuristics. Here, the algorithm is based on the assumption that connected vascular structures will be of small diameter when they finally reach the lesion. Furthermore, the lesion must not infiltrate any connected tissue. The segmentation pipeline will be robust enough to ignore connections of the lesion to tissue and will also ignore connected vessel systems, in this case the pulmonary artery and vein.

To obtain a first look onto the example in MeVisLab, please open the network file `Lesion Segmentation.mlab` [6]. In the left 2-D view, scroll to slice number 46 using the scroll wheel of your mouse while pointing at the 2-D view. Place a marker into the lesion as seen in Figure 7. The segmented lesion will be instantly highlighted in the 2-D as well as in the 3-D views. To display the maximum diameter of the lesion, open the module panel of the `MaxDistance` module.

```

1 Window {
2   style = _default
3
4   title = "Vessel Segmentation"
5
6   Horizontal {
7     expandY = yes
8     Box "Input" { expandY = yes layout = Vertical
9       Horizontal {
10        HyperLabel { text = "Scroll to a slice in which the vascular structures
11          you would like to segment are visible and click into
12            the center of one or more vascular structures." }
13        Button MarkerEditor.deleteAll { title = "Delete All Markers" }
14      }
15      Viewer 2DView.self { mw = 400 mh = 400 type = SoRenderArea }
16    }
17    Box "Output" { expandY = yes layout = Vertical
18      Viewer 3DView.self { mw = 400 }
19    }
20  }
21 }
```

FIGURE 6 The UI script defines which parameters of the image-processing network are visible and how they are presented to the user.

An isosurface represents regions of voxels of a certain value.

For an inspection of intermediate results in the image-processing pipeline, display the Output Inspector by choosing View → Views → Output Inspector from the menu bar. The Output Inspector will display the data available at a selected module connector.

The segmentation and quantification of the lesion involves the following steps:

- 1) *Preprocessing*: Thresholding and connected components analysis to compute a mask of the lung image data.
- 2) *Segmentation*: Region growing using an initial seed point.
- 3) *Postprocessing*: Dilation and computation of the border of the lesion mask image.
- 4) *Visualization*: Direct volume rendering of the image data and highlighting of the lesion.
- 5) *Measurement*: Quantification of the lesion's maximum diameter using the lesion mask image.

Large parts of a chest CT image data will contain air voxels. Air and tissue can be well distinguished in CT image data because of their different voxel values. We use an `IntervalThreshold` module with a threshold value of -200 HU for the separation since air voxel values are around $-1,000$ HU and tissue values are more than 0 HU. Separation of the lesion from the connected tissue is more challenging; the voxel values may not differ enough to prevent the region growing from spreading from the lesion into the adjacent regions. Since we know that the lesion will be inside the lung, we can use a mask of the lung volume to cut off the lesion at the border. This brings us the task of calculating a lung volume mask.

We already obtained a mask of all nontissue voxels from the `IntervalThreshold` module. Assuming the lung will

be the single largest object in the images, an assumption truly holding for a chest CT acquisition, a connected component analysis using the `ConnectedComponents` module will find this single largest cluster. The resulting mask image of this cluster will still contain holes and notches from the blood vessels, bronchial tubes, free-standing as well as wall-attached lesions of the lung. We are particularly interested in the boundary of the volume, in the convex hull of the lung. The `ConvexHull` module computes the convex hull for voxels of a specific value. Now that we possess a mask of tissue plus lesion and convex hull of the lung, we create a new mask of all nonair voxels that are contained within the lung using a `Mask` module. The resulting mask will still contain vascular structures. In contrast to the lesions, vascular structures are of elongated shape, a shape we eliminate from the images using a fundamental, morphological operation called *opening*. Opening uses a structural element, called the *kernel*, to eliminate structures smaller than this structural element. Hence, the kernel size parameter in the `FastMorphology` module needs to be as large as the largest structures we would like to remove. The kernel will have a global effect on the mask image and also modify the morphology of lesions; borders will be smoother and small notches will disappear. This is something to keep in mind when applying these operations.

If there is only one lesion in the data set, we can stop now. The mask image will only contain this lesion. However, more often than not, the images will reveal multiple lesions. To quantify

the size of a specific lesion, we opt for an interactive selection. Hence, the `RegionGrowing` module in the image pipeline will use a marker, placed on a 2-D slice, to narrow down the image mask to just the selected lesion.

The calculation of the diameter is a perfect example of using the large image processing library of MeVisLab. There is no module that computes the maximum diameter of a 3-D image object from a given mask image. One could decide to implement such a module. However, MeVisLab provides a module to calculate the maximum distance between a set of markers called `XMarkerListMaxDistance`. Fortunately, we can convert a mask image into a set of markers using the module `MaskToMarkers`. However, applying this module directly to our lesion mask image would create many inside markers, while all we need are markers on the outer border of the lesion. The `Surround` module offers a mode to calculate just the boundary of a set of voxels. Placing this module between the `RegionGrowing` and `MaskToMarkers` modules saves computation time and enables us to overlay the resultant border image onto the original image in the 2-D view to provide visual feedback about the segmentation to the user.

Strictly speaking, the 3-D visualization of the lesion is only of limited diagnostic advantage. It provides an additional means for the inspection of the segmentation result, hence the calculation of the maximum diameter. However, depending on the chosen therapy, which could, for instance, perform local

Automatic segmentation methods are often restricted to a specific type of object and data on which they operate because of need for heuristics.

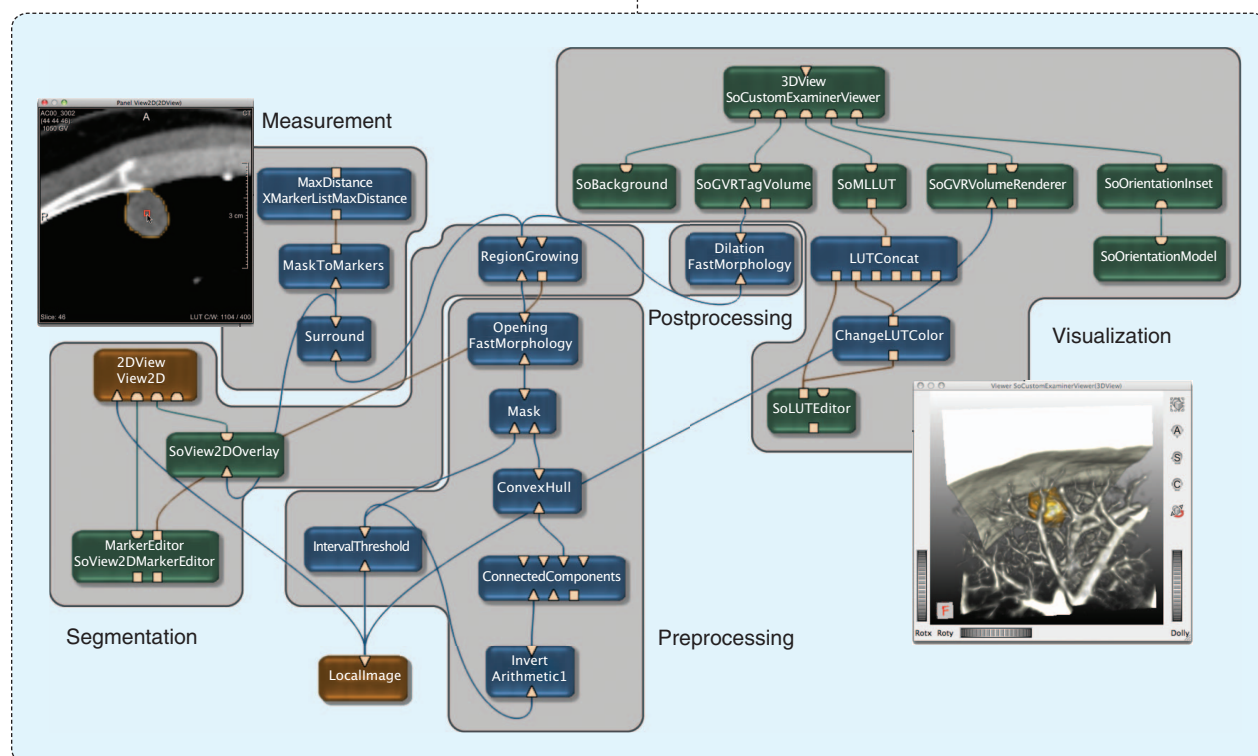


FIGURE 7 The image-processing network used to quantify the maximum diameter of a wall-connected lesion in the lung.

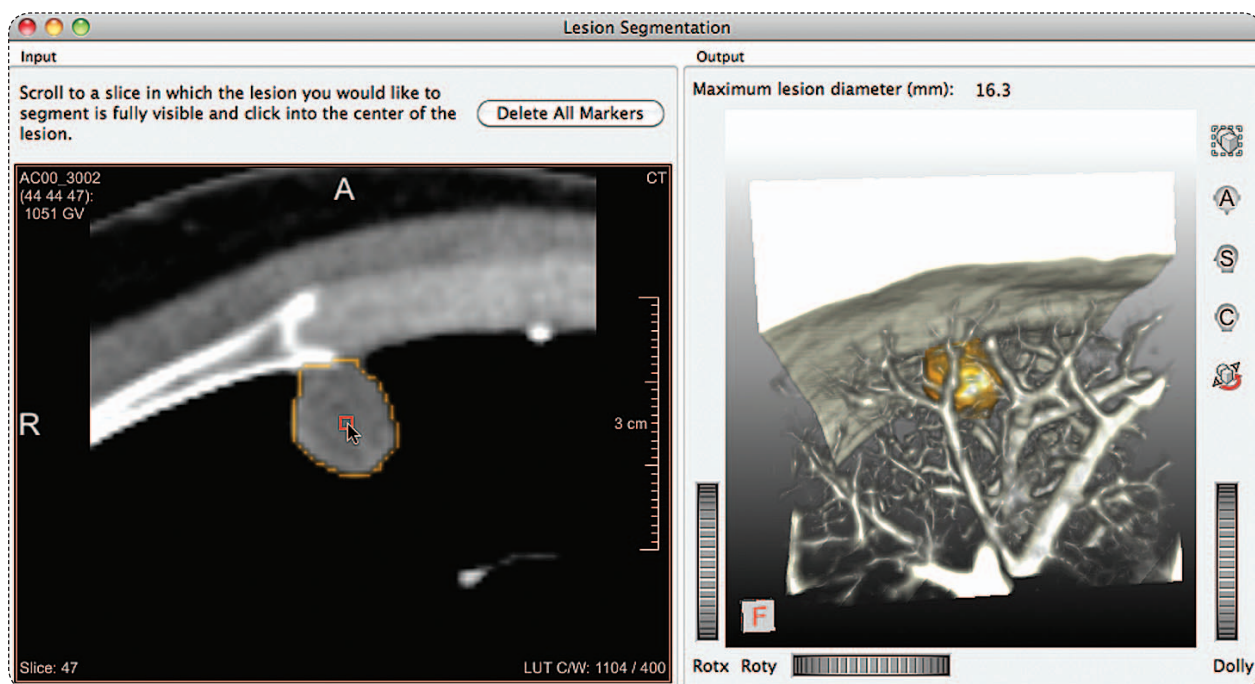


FIGURE 8 The final UI of the lesion quantification example.

destruction of the tumor, knowledge about the lesion's surroundings could prove beneficial.

Direct volume rendering is well suited for the 3-D depiction of this kind of data. Since air and tissue are represented by different values in CT and MR images, visual separation does not need an explicit mask or any additional parameters for the transfer function (e.g., derived values, such as the gradient or curvature). Highlighting the lesion itself is different. We employ the slightly enlarged lesion mask image to apply a different transfer function to the voxel values of the lesion in the original image. Voxel values with a mask value of zero are rendered using the first transfer function, designed using the *SoLUTEditor* module, whereas the second transfer function is applied to voxel values with a mask value of one. This transfer function is derived from the first by means of the *ChangeLUTColor* module. It overrides just the color values of the transfer function with a highlighting color. The volume rendering itself is implemented in the *SoGVRVolumeRenderer* module, which on its own offers various parameters to tune the visualization. The final UI of our lesion segmentation example can be seen in Figure 8.

Visualization and analysis often involve a separation of structures.

Advanced Image Processing

In this tutorial, we could only scratch the surface of medical image processing. Two more advanced image-processing topics will conclude this tutorial and hopefully spark some interest in this field. There is still a lot to explore.

Object-Based Image Analysis

Even the most sophisticated image analysis methods available today cannot cope with the remarkable pattern-recognition

capabilities of human beings. For a human mind, an image represents a complex web of interrelated objects at different scales. It is prior knowledge on the characteristic properties and mutual relations of these image objects that enables a human to understand the image content. For a computer, an image is merely a rectangular grid of voxels with intensity values. Therefore, most image-processing algorithms are limited to the evaluation of voxel values in local neighborhoods. To overcome this limitation, a rich set of tools for analyzing images on the basis of objects has been created using the

MeVisLab platform, similar to the way a human understands an image. This set comprises simple-to-use functionality for extracting spectral, shape, and relational object properties from the image and for the knowledge-based evaluation of this information to identify the structures of interest.

In many areas of medical imaging, object-based image analysis solves problems that are unsolvable by ordinary voxel-based image analysis. Complex image structures often do not become instantly obvious. Instead, image understanding is an iterative process of deriving information from the image and applying prior knowledge. This iterative process is demonstrated by an application for the detection of the spine in CT images [Figure 9(a)]. Voxel-based image-processing algorithms fail in this application because the spine does not exhibit unique intensity values. Also, many are too distorted or inconspicuous to be identified in isolation. Object-based image analysis solves this problem by exploiting prior knowledge that the spine is a loose chain of vertically aligned vertebrae with characteristic spectral and shape properties. A watershed-based oversegmentation creates the initial set of objects [Figure 9(b)]. The

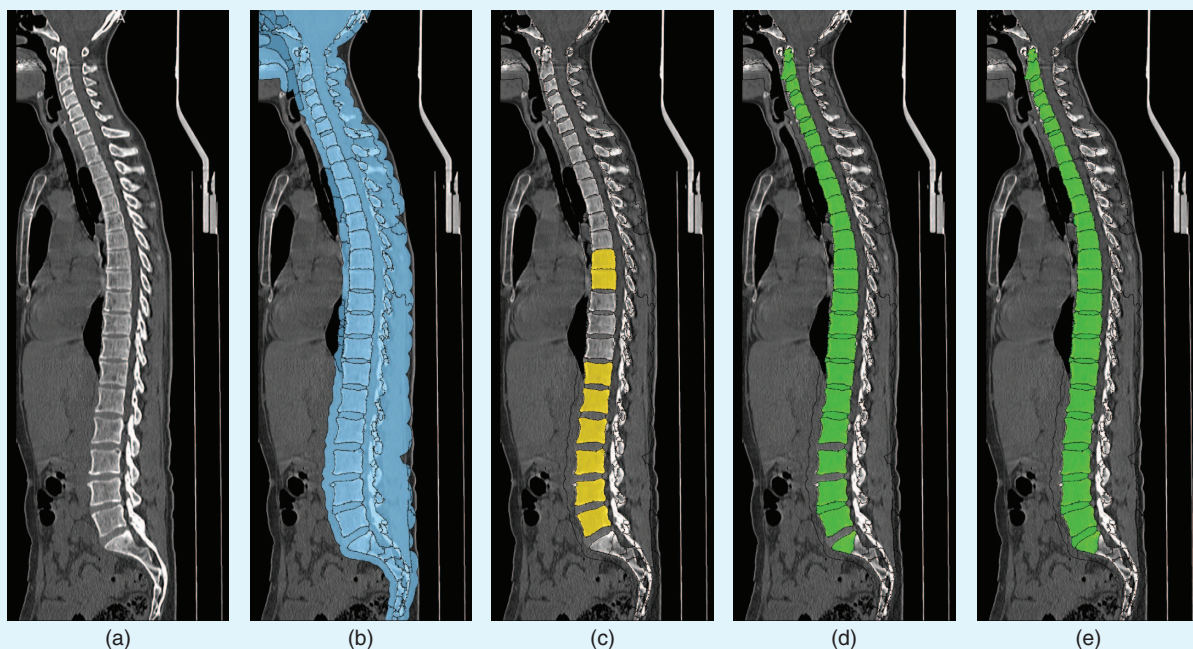


FIGURE 9 Object-based detection of the spine: (a) original image, (b) initial oversegmentation, (c) detection of certain vertebrae, (d) iterative classification of adjacent objects leading to full spine detection, and (e) refinement to close possible gaps.

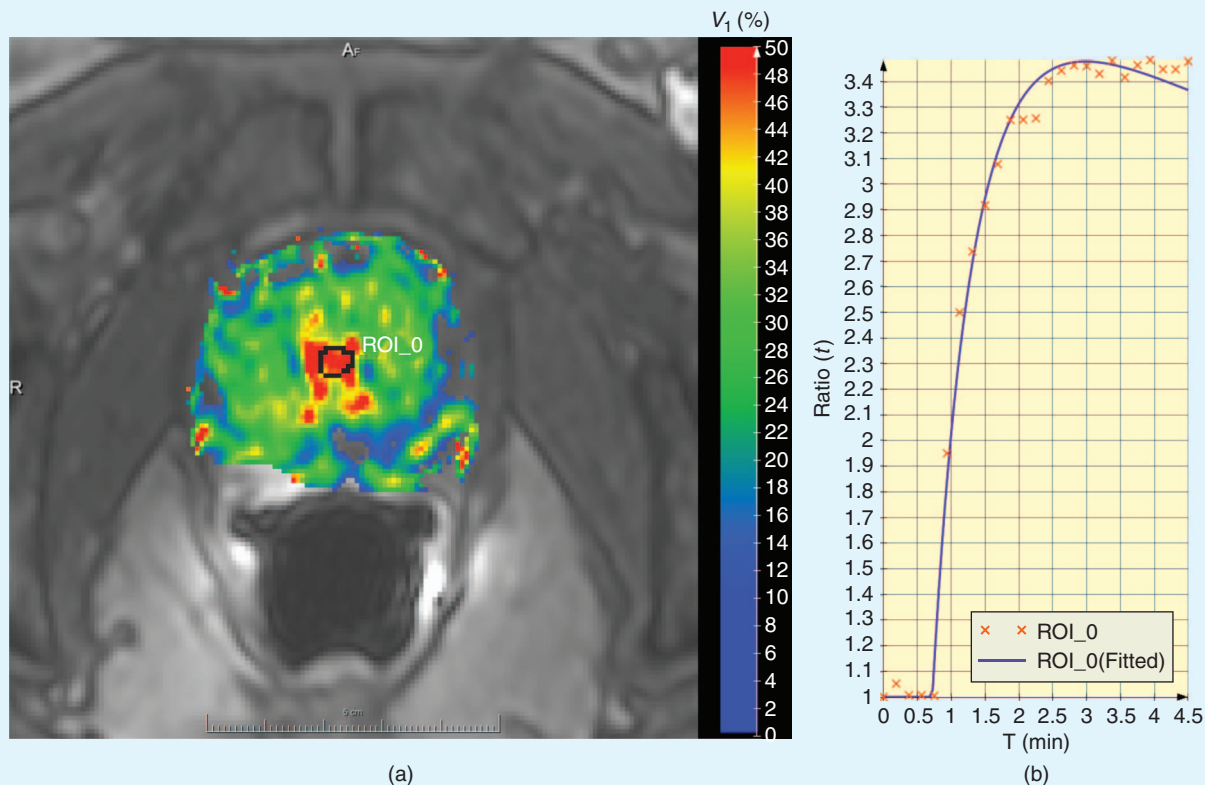


FIGURE 10 (a) Imaging the vasculature of a tumor with the help of a contrast agent. (b) The distribution of the contrast agent is measured periodically over time (red crosses), and afterward a model curve (blue curve) is fitted to the measured data. The model provides vascular information on the tissue and is superimposed onto the anatomical image data of the suspicious region as a color map.

first step of the analysis is to identify a few of these objects as vertebrae by employing strong constraints on their spectral and shape properties [Figure 9(c)]. Adjacent vertebrae, whose visual properties alone do not allow a certain classification, can now be identified by their relative location to already detected vertebrae. Repeating the last step until the whole spine is recognized yields a robust detection method that even works with noisy images and vertebrae affected by lesions [Figure 9(d)]. Remaining gaps (due to an inappropriate object presegmentation) can be closed with a subsequent refinement step [Figure 9(e)].

While we usually possess a wealth of knowledge about the objects in an image, it is often difficult to put this knowledge into words. This applies, particularly, when the knowledge has to be stated numerically, for instance, as the expected mean intensity or size of certain image objects. For this reason, our object-based image analysis methods are capable to learn. All the user has to do is to provide examples of relevant image objects, from which the computer then derives the necessary knowledge to identify further objects automatically.

Dynamic Imaging

Today, medical imaging offers a wide variety of functional imaging that can give information on human tissue beyond that on shape and structure available from the imaging information alone. These functional imaging processes can yield valuable parameters on blood supply and circulation, and oxygen consumption or cell metabolism. The parameters find a broad application in diagnosis and treatment monitoring of various diseases.

Frequently, these parameters are obtained by acquiring subsequent images over time after injecting a contrast agent into the bloodstream of the patient. Depending on the physical and biological properties of the contrast agents, the enhancement course and pattern of the contrast agent will yield information on tissue properties. For example, Fludeoxyglucose-18 is processed in cells in the same manner as glucose and therefore, is a good marker for cell metabolism. This can be exploited to measure the metabolism of tumors and differentiate tumor tissue from healthy tissue. Other contrast agents are kept confined in the blood vessels and therefore, their temporal and spatial distribution is related to blood volume and flow.

Once data are acquired, the time-to-signal-curve can be derived from the data, either by recording the signal changes over time for single voxels or averaging over regions of interest (ROI) to obtain a mean time-to-signal curve. Each curve then can be evaluated using signal-processing techniques. Commonly used are compartment models describing the processing or distribution of the contrast agent by the tissue in the voxel or ROI. The results of this analysis can either be a set of parameter maps displaying the derived parameters for each voxel or, in the ROI-based version, a table containing the parameters

for the ROI. An example is shown in Figure 10, displaying the results for a very commonly used method—the dynamic contrast-enhanced magnetic resonance imaging (DCE-MRI)—where a paramagnetic contrast agent is used to evaluate the vasculature of tumors.

MeVisLab allows the user to import series of volumes acquired over time of a number of medical imaging modalities, for instance MRI, CT, and PET. There are modules available to select an ROI in the images and generate time-intensity curves for these regions. Figure 10 shows an example evaluation created with MeVisLab. In Figure 10(a), a curve generated from the DCE-MRI data is displayed. It shows the temporal course of the ratio of signals between

the contrast-agent-free first time period and time in which the contrast agent has arrived in the tissue. The ratio is proportional to the contrast agent concentration and can be used to optimize the parameters of the model describing this time course. In this case, the model describes the time course of the ratio in dependence on some basic model parameters: the vessel permeabilities and interstitial volume. The parameters are determined for all voxels in a dynamic data set by optimizing the model to the data using a least-square algorithm. After conversion by a color map,

these parameters can be superimposed on top of the original image data as shown in Figure 10.

A series of DICOM images, which is the most basic form for medical images, can be retrieved from a disk using the DirectDicom Import module.

Acknowledgments

The authors wish to thank Dieter Haemmerich for fruitful discussions on the structure and contents of the tutorial and the MeVisLab team at MeVis Medical Solutions for exceptional support.

Felix Ritter, André Homeyer, Hendrik Laue, Michael Schwier, and Heinz-Otto Peitgen are with Fraunhofer MEVIS—Institute for Medical Image Computing. Tobias Boskamp and Florian Link are with MeVis Medical Solutions AG.

References

- [1] MeVisLab. [Online]. Available: <http://www.mevalab.de/>
- [2] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 3rd ed. Englewood Cliffs, NJ: Prentice-Hall, 2007
- [3] A. P. Dhawan, H. K. Huang, and D.-S. Kim, *Principles and Advanced Methods in Medical Imaging and Image Analysis*, 1st ed. Singapore: World Scientific, 2008.
- [4] C. D. Hansen and C. R. Johnson, *The Visualization Handbook*. New York: Elsevier, 2005.
- [5] A. Homeyer, M. Schwier, and H. K. Hahn, A generic concept for object-based image analysis," in *Proc. Int. Conf. Computer Vision Theory and Applications*, 2010, pp. 530–533.
- [6] F. Ritter. Example data of the article. [Online]. Available: http://www.mevalab.de/IEEE_Pulse/